

Campus Ride Management Platform

Design Document

Real-Time Campus Mobility and Ride Management Platform

Technology Stack

Java 21 · Spring Boot 3 · PostgreSQL · React + TypeScript + Vite · WebSocket/STOMP · Leaflet / OpenStreetMap

1. Problem Understanding

IIT Roorkee's large campus depends on e-rickshaws for last-mile mobility. Without centralized coordination, passengers have limited visibility into available drivers, drivers have limited visibility into demand, and ride state changes are hard to track.

- Centralize passenger ride requests, driver availability, ride assignment, lifecycle tracking, and feedback.
- Use real-time communication to synchronize request, assignment, status, availability, and driver location changes.
- Keep the system maintainable for a small team while preserving the core engineering concerns of ride-hailing systems.

2. System Architecture

The platform follows a three-tier architecture: React SPA frontend, Spring Boot REST/WebSocket backend, and PostgreSQL persistence layer.

Layer	Technology	Responsibility
Frontend	React 18, TypeScript, Vite, Leaflet	Role-specific dashboards, forms, map rendering, local session storage, STOMP client.
Backend API	Java 21, Spring Boot 3, Spring Security	REST endpoints, validation, JWT-authenticated API access, business workflow orchestration.
Real-Time	Spring WebSocket + STOMP	Topic-based updates for ride requests, assignment, lifecycle, driver availability, dashboard, and GPS events.
Database	PostgreSQL, JPA/Hibernate	Durable storage for users, profiles, vehicles, rides, rejections, ratings, and campus locations.

2.1 Component Interaction

- HTTP REST is used for request/response operations such as login, profile updates, ride creation, lifecycle actions, dashboard queries, and ratings.
- WebSocket/STOMP on /ws is used for push events. The current security config permits /ws, while REST endpoints are protected by JWT. Production hardening can add authenticated WebSocket handshakes.
- Vite proxies /api and /ws to the backend on localhost:8099 during local development.

3. Database Schema

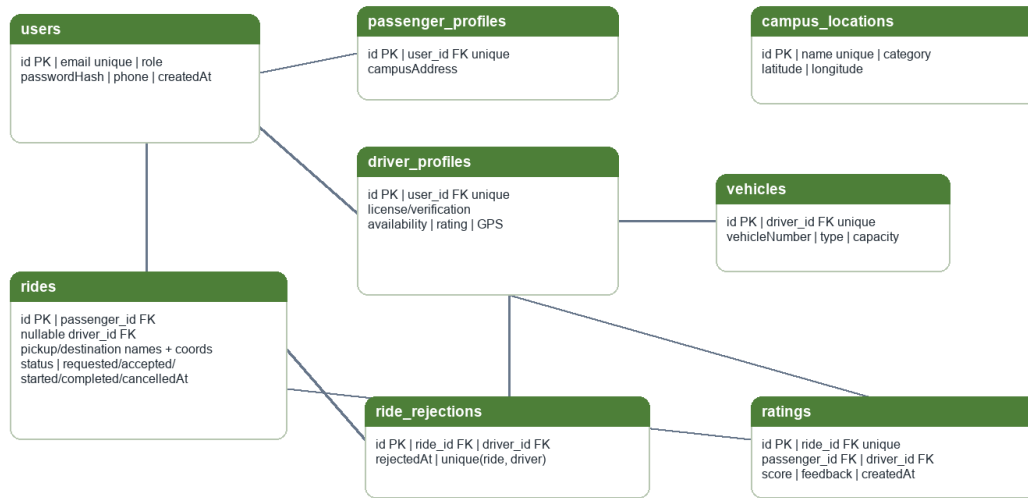
Hibernate ddl-auto update creates and evolves tables during development. The schema separates shared identity from passenger and driver details, and keeps ride history, rejections, ratings, and campus locations in separate tables.

Table	Key Columns	Notes
users	id, name, email unique, phone, passwordHash, role, createdAt	Base identity for passenger and driver accounts.
passenger_profiles	id, user_id unique FK, campusAddress	Passenger-specific profile extension.
driver_profiles	id, user_id unique FK, licenseNumber, verificationDocument, verificationStatus, availabilityStatus, averageRating, ratingCount, currentLatitude/currentLongitude	Driver verification, availability, rating summary, and live GPS state.
vehicles	id, driver_id unique FK, vehicleNumber, vehicleType, capacity	One vehicle record per driver.
rides	id, passenger_id FK, driver_id nullable FK, pickupLocation, destination, pickup/destination lat/lng, status, requestedAt, acceptedAt, startedAt, completedAt, cancelledAt, version	Central ride lifecycle record. Pickup/destination are stored as names and coordinates, not campus_location foreign keys.
ride_rejections	id, ride_id FK, driver_id FK, rejectedAt, unique(ride_id, driver_id)	Prevents rides rejected by a driver from repeatedly appearing to that same driver.
campus_locations	id, name unique, category, latitude, longitude	Editable campus points used to seed ride coordinates.
ratings	id, ride_id unique FK, passenger_id FK, driver_id FK, score, feedback, createdAt	One passenger rating per completed ride.

4. Entity Relationship Diagram

The ERD below reflects the implemented JPA relationships. Rides point to users for passenger and driver. Campus locations are lookup/seed data for coordinates, not ride foreign keys.

Entity Relationship Diagram



users is the common identity table. Rides reference users for passenger/driver. Campus locations seed coordinates but are not ride foreign keys.

5. API Overview

Method	Endpoint	Purpose
POST	/api/auth/register/passenger	Register passenger and return JWT.
POST	/api/auth/register/driver	Register driver, vehicle, verification profile, and return JWT.
POST	/api/auth/login	Authenticate user and return JWT.
GET/PUT	/api/users/me	Read or update authenticated user profile.
POST	/api/drivers/availability/online	Set driver ONLINE, or BUSY if already assigned.
POST	/api/drivers/availability/offline	Set driver OFFLINE when no active ride exists.
GET	/api/drivers/available	List currently ONLINE drivers.
GET	/api/drivers/requests	List requested rides not rejected by this driver.
GET	/api/drivers/dashboard	Return active ride, stats, ride history, ratings, and chart data.
POST	/api/drivers/location	Store driver GPS and broadcast location events.
GET	/api/locations	Return campus pickup/destination points.
POST/GET	/api/rides, /api/rides/my	Create ride and fetch user ride history.
GET	/api/rides/{rideId}	Return a ride visible to participant or requested driver pool.
POST	/api/rides/{id}/accept reject start complete cancel	Ride assignment and lifecycle transitions.
POST	/api/ratings	Submit rating/feedback for a completed ride.

5.1 WebSocket Topics (STOMP)

- /topic/rides/requests, /topic/rides/{rideId}, and /topic/rides/{rideId}/driver-location publish ride request, ride status, and ride-specific location events.
- /topic/users/{userId}/notifications sends per-user passenger/driver ride notifications.
- /topic/drivers/availability and /topic/drivers/location publish driver pool and GPS updates.
- /topic/drivers/{driverId}/dashboard pushes assigned ride and dashboard updates to the driver.

6. Design Decisions

Decision	Reason
WebSocket/STOMP over polling	Push updates are better for ride requests, assignment notifications, availability changes, and live driver movement.
Pessimistic row lock on ride acceptance	acceptRide uses findByIdForUpdate inside a transaction. If the ride is no longer REQUESTED or already has a driver, the backend returns a bad-request error.
JWT stateless REST authentication	REST APIs are protected by JWT and BCrypt password hashes. Current /ws endpoint is permitted; production can authenticate the handshake.
Simple React state	useState/useEffect are enough for the current app; adding Redux/Zustand would add overhead without clear benefit.
Leaflet/OpenStreetMap	No billing/API key burden; supports campus pickup, destination, and assigned driver markers.
Database-backed campus locations	Coordinates can be corrected in PostgreSQL/pgAdmin without changing frontend code.
Driver GPS throttling	Frontend sends location when enough time or movement has passed, roughly 10 seconds or 12 meters, avoiding noisy updates.

7. Feature Completeness Summary

Feature	Status
User authentication, JWT, role-based REST access	Done
Passenger and driver registration/profile management	Done
Driver vehicle and verification information	Done
Driver availability: OFFLINE, ONLINE, BUSY	Done
Ride request, accept, reject, start, complete, cancel	Done
Single-driver assignment using DB locking	Done
Ride lifecycle: REQUESTED, ACCEPTED, IN_PROGRESS, COMPLETED, CANCELLED	Done
Driver dashboard with summary cards, chart, history, ratings	Done
Ratings and written feedback	Done
Live map with Leaflet/OpenStreetMap	Done
Driver GPS tracking and realtime marker updates	Done
Ride scheduling, digital payments, demand analytics, demand forecasting	Not implemented yet / future optional scope